

Laboratório JavaScript: Painel de Servidores

Programação Front-end | Prof. Renan

Relatório de evidências da atividade guiada

Aluno	Sonny Heros Campos Cardoso
Matrícula	202510787
Turma	3AM
Início	30/09/2026 08:46
PDF gerado em	30/09/2026 10:46
Verificação	AE4F8F8C5BA8B9AD

Parâmetros individuais

Servidores	7
Prefixo	srv-787
Limite alerta	55%
Limite crítico	75%
Redução (while)	20

Resumo das etapas

1. Preparando o projeto	HTML, CSS e classes 2 print(s) concluída 30/09/2026 09:06
2. Modelando servidores com objetos	Objeto e array 1 print(s) concluída 30/09/2026 09:27
3. Classificando a carga com uma função	function, if, else if, else 1 print(s) concluída 30/09/2026 09:35
4. Desenhando o painel com for	Laço for e DOM 1 print(s) concluída 30/09/2026 09:50
5. Trocando classes pelo JavaScript	classList e eventos 2 print(s) concluída 30/09/2026 10:06
6. Estabilizando servidores com while	Laço while 2 print(s) concluída 30/09/2026 10:18
7. Integração e entrega final	Revisão geral 1 print(s) concluída 30/09/2026 10:34

Declaração: o aluno declarou que o código, os prints e as respostas foram produzidos por ele durante esta atividade.

Etapa 1: Preparando o projeto

Conceito: HTML, CSS e classes | Concluída em 30/09/2026 09:06

Montar a estrutura do painel e definir as classes CSS que o JavaScript vai ligar e desligar nas próximas etapas.

Conferência

- [x] Criei a pasta com index.html, style.css e script.js
- [x] Preenchi as classes .ok, .alerta, .critico e body.manutencao
- [x] A página abre no navegador e o console não mostra erros

Pergunta

Por que a tag <script> fica no final do <body>? O que aconteceria com document.getElementById se ela estivesse no <head> sem o atributo defer?

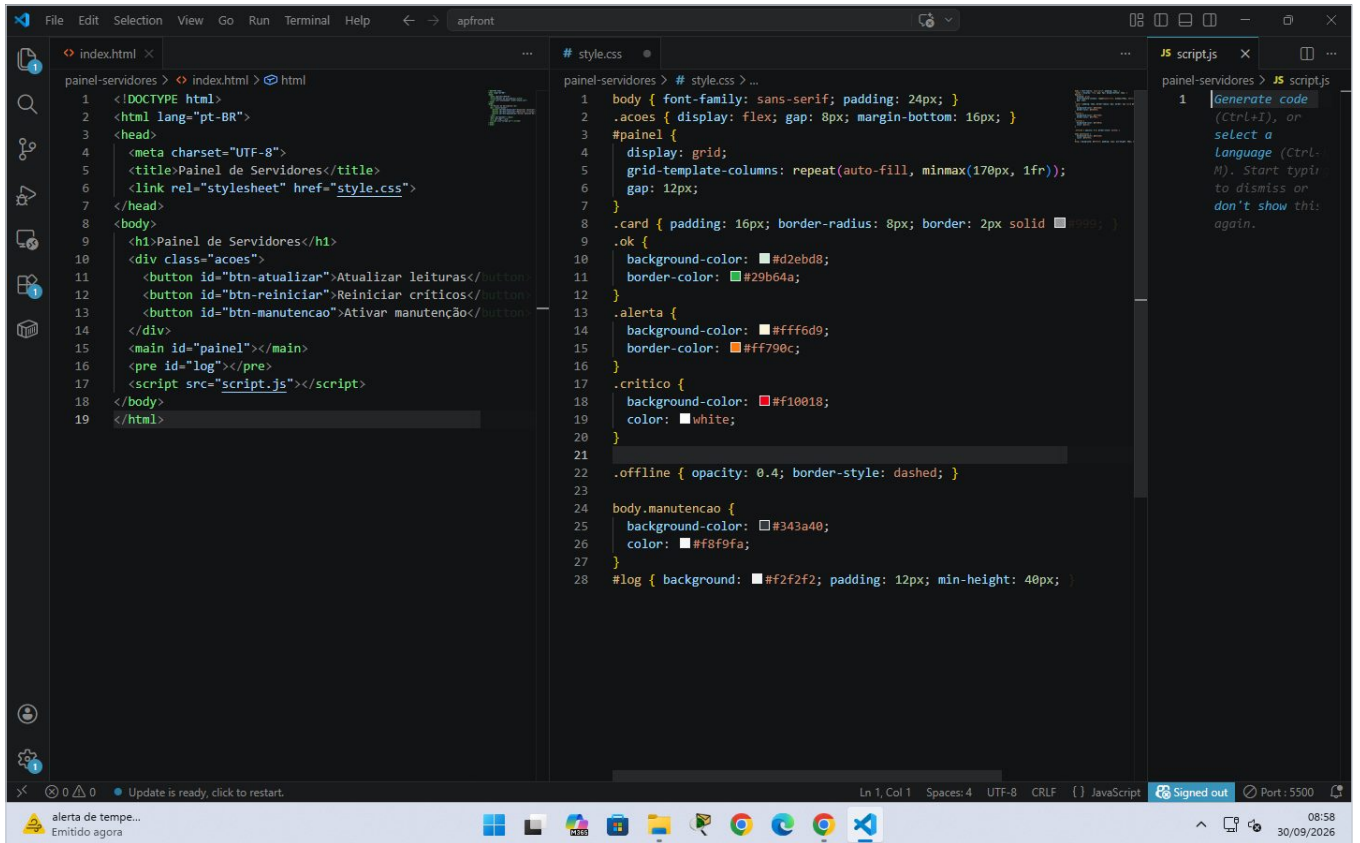
A tag <script> deve ficar no final do <body> como forma de garantia de que todo o conteúdo do HTML seja carregado e renderizado na tela antes do JavaScript ser executado, pois, se o script estivesse no <head>, o navegador tentaria executar o JS antes de ler os elementos da página, fazendo com que o comando "document.getElementById" retorne um erro ou nulo, pois tentaria buscar um ID de botão que ainda não existe na memória do navegador.

Cole o conteúdo final do seu style.css

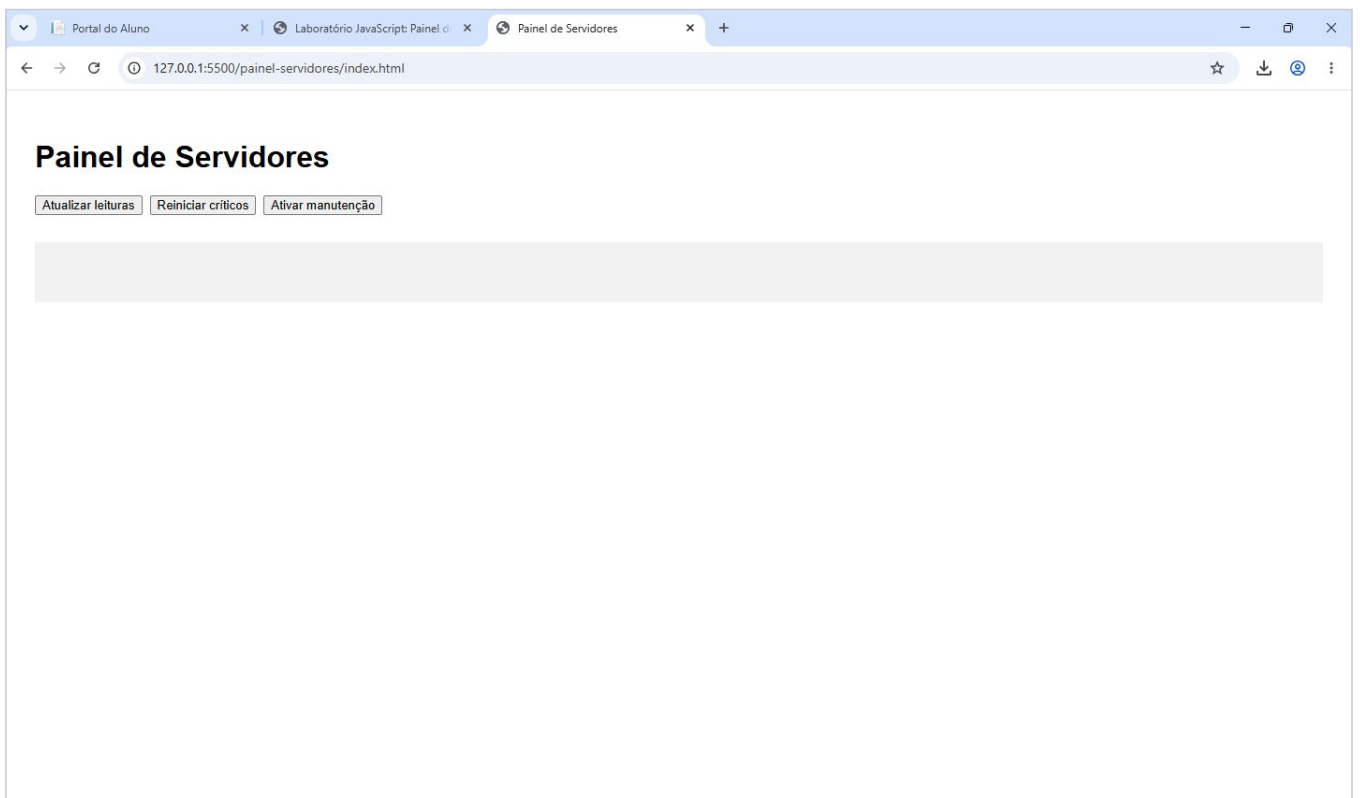
```
body { font-family: sans-serif; padding: 24px; }
.acoes { display: flex; gap: 8px; margin-bottom: 16px; }
#painel {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(170px, 1fr));
  gap: 12px;
}
.card { padding: 16px; border-radius: 8px; border: 2px solid #999; }
.ok {
  background-color: #d2ebd8;
  border-color: #29b64a;
}
.alerta {
  background-color: #fff6d9;
  border-color: #ff790c;
}
.critico {
  background-color: #f10018;
  color: white;
}
.offline { opacity: 0.4; border-style: dashed; }

body.manutencao {
  background-color: #343a40;
  color: #f8f9fa;
}
#log { background: #f2f2f2; padding: 12px; min-height: 40px; }
```

Prints



Print 1 de 2 | Captura de tela 2026-09-30 085812.png | anexado em 30/09/2026 09:06



Print 2 de 2 | Captura de tela 2026-09-30 090108.png | anexado em 30/09/2026 09:06

Etapa 2: Modelando servidores com objetos

Conceito: Objeto e array | Concluída em 30/09/2026 09:27

Representar cada servidor como um objeto e guardar os 7 servidores da sua infraestrutura em um array.

Conferência

- [x] O array tem exatamente 7 servidores
- [x] Os nomes seguem o padrão srv-787-1 até srv-787-7
- [x] Há cpu abaixo de 55, entre 55 e 74, e de 75 ou mais
- [x] Pelo menos um servidor está com online: false
- [x] O console.table mostra a tabela no console

Pergunta

Qual a diferença entre `servidores[0].cpu` e `servidores[0]["cpu"]`? Dê um exemplo em que a notação de colchetes é obrigatória.

As duas formas acessam a mesma propriedade do objeto. A forma com 'ponto' é mais direta e mais comum, mas a forma com colchetes é obrigatória quando o nome da propriedade tem espaços, caracteres especiais, ou quando o nome da chave está armazenada em uma variável dinâmica.

Ex.: `let chave = 'cpu'; console.log(servidores[0][chave]);`

Cole o array servidores completo

```
const servidores = [  
  { nome: "srv-787-1", ip: "10.0.0.1", cpu: 35, online: true },  
  { nome: "srv-787-2", ip: "10.0.0.2", cpu: 50, online: true },  
  { nome: "srv-787-3", ip: "10.0.0.3", cpu: 60, online: true },  
  { nome: "srv-787-4", ip: "10.0.0.4", cpu: 70, online: true },  
  { nome: "srv-787-5", ip: "10.0.0.5", cpu: 75, online: true },  
  { nome: "srv-787-6", ip: "10.0.0.6", cpu: 90, online: true },  
  { nome: "srv-787-7", ip: "10.0.0.7", cpu: 0, online: false }  
];  
  
console.table(servidores);  
console.log(servidores[0].nome);  
console.log(servidores[0]["cpu"]);
```

Prints

Painel de Servidores

127.0.0.1:5500/painel-servidores/index.html

Painel de Servidores

Atualizar leiturasReiniciar criticosAtivar manutenção

DevTools is now available in Portuguese

Don't show againAlways match Chrome's language

Switch DevTools to Portuguese

ElementsConsoleSourcesNetwork

topFilterDefault levelsNo issues

(index)	nome	ip	cpu	online
0	'srv-787-1'	'10.0.0.1'	35	true
1	'srv-787-2'	'10.0.0.2'	50	true
2	'srv-787-3'	'10.0.0.3'	60	true
3	'srv-787-4'	'10.0.0.4'	70	true
4	'srv-787-5'	'10.0.0.5'	75	true
5	'srv-787-6'	'10.0.0.6'	90	true
6	'srv-787-7'	'10.0.0.7'	0	false

Array(7)
srv-787-1
35

29°C
Ensolarado

09:27
30/09/2026

Print 1 de 1 | Captura de tela 2026-09-30 092749.png | anexado em 30/09/2026 09:27

Etapa 3: Classificando a carga com uma função

Conceito: function, if, else if, else | Concluída em 30/09/2026 09:35

Escrever a função classificar, que recebe a cpu e devolve "ok", "alerta" ou "critico" usando os seus limites (55 e 75).

Conferência

- [x] A função retorna "invalido" para valores fora de 0 a 100
- [x] Os quatro testes no console mostram exatamente os resultados esperados
- [x] Usei if, else if e else na mesma função

Pergunta

Por que a ordem das condições importa na sua função? Explique o que aconteceria com classificar(75) se o teste de alerta viesse antes do teste de crítico.

A ordem das condições tem importância porque o código é executado de cima pra baixo e interrompe a verificação assim que encontra a primeira condição verdadeira. Caso o teste de alerta venha antes do teste crítico, uma chamada para '(classificar(75))' entraria na primeira condição, pois 75 é maior ou igual a 55, o que causaria um erro na lógica de monitorização.

Cole a função classificar completa

```
const LIMITE_ALERTA = 55;
const LIMITE_CRITICO = 75;

function classificar(cpu) {
  if (cpu < 0 || cpu > 100) {
    return "invalido";
  } else if (cpu >= LIMITE_CRITICO) {
    return "critico";
  } else if (cpu >= LIMITE_ALERTA) {
    return "alerta";
  } else {
    return "ok";
  }
}

console.log(classificar(54));
console.log(classificar(55));
console.log(classificar(75));
console.log(classificar(101));
```

Prints

Painel de Servidores

127.0.0.1:5500/painel-servidores/index.html

Painel de Servidores

Atualizar leiturasReiniciar criticosAtivar manutenção

DevTools is now available in Portuguese

Don't show againAlways match Chrome's language

Switch DevTools to Portuguese

ElementsConsoleSourcesNetwork

topFilterDefault levelsNo issues

(index)	nome	ip	cpu	online
0	'srv-787-1'	'10.0.0.1'	35	true
1	'srv-787-2'	'10.0.0.2'	50	true
2	'srv-787-3'	'10.0.0.3'	60	true
3	'srv-787-4'	'10.0.0.4'	70	true
4	'srv-787-5'	'10.0.0.5'	75	true
5	'srv-787-6'	'10.0.0.6'	90	true
6	'srv-787-7'	'10.0.0.7'	0	false

Array(7)

srv-787-1script.js:12

35script.js:13

okscript.js:31

alertascript.js:32

criticascript.js:33

invalidoscript.js:34

29°C Ensolarado

09:34 30/09/2026

Print 1 de 1 | image.png | anexado em 30/09/2026 09:34

Etapa 4: Desenhando o painel com for

Conceito: Laço for e DOM | Concluída em 30/09/2026 09:50

Percorrer o array com um for tradicional, criar um card para cada servidor e aplicar a classe CSS correspondente à carga.

Conferência

- [x] Cada servidor aparece como um card na página
- [x] A cor de cada card corresponde à classificação
- [x] Servidores offline aparecem esmaecidos
- [x] O log mostra a quantidade correta de servidores críticos

Pergunta

Explique as três partes do seu for (inicialização, condição e incremento) usando o seu próprio código. Por que usamos $i < servidores.length$ e não $i \leq servidores.length$?

No código 'for (let i = 0; i < servidores.length; i++)':

A inicialização é: let i = 0 (o contador começa em zero)

A condição é: i < servidores.length (o laço repete enquanto 'i' for menor que 7)

O incremento é i++ (soma 1 a cada volta)

Usei < em vez de <= porque os arrays em JS começam no índice 0. Como temos 7 servidores, o último está no índice 6. Se usássemos <=, o laço tentaria acessar o índice 7, que não existe (daria undefined), e quebraria o código na hora de ler a CPU.

Cole a função renderizar completa

```
const painel = document.getElementById("painel");
const log = document.getElementById("log");

function renderizar() {
  painel.innerHTML = "";
  let criticos = 0;

  for (let i = 0; i < servidores.length; i++) {
    const s = servidores[i];
    const card = document.createElement("div");
    card.classList.add("card");

    const classificacao = classificar(s.cpu);

    card.classList.add(classificacao);

    if (s.online === false) {
      card.classList.add("offline");
    }

    if (classificacao === "critico") {
      criticos++;
    }

    card.innerHTML = "<strong>" + s.nome + "</strong><br>" + s.ip + "<br>CPU: " + s.cpu + "%";
    painel.appendChild(card);
  }

  log.textContent = "Servidores críticos: " + criticos;
}

renderizar();
```

Prints

Painel de Servidores

127.0.0.1:5500/painel-servidores/index.html

Painel de Servidores

Atualizar leiturasReiniciar críticosAtivar manutenção

srv-787-1
10.0.0.1
CPU: 35%

srv-787-2
10.0.0.2
CPU: 50%

srv-787-3
10.0.0.3
CPU: 60%

srv-787-4
10.0.0.4
CPU: 70%

srv-787-5
10.0.0.5
CPU: 75%

srv-787-6
10.0.0.6
CPU: 90%

srv-787-7
10.0.0.7
CPU: 0%

Servidores críticos: 2

DevTools is now available in Portuguese

Don't show againAlways match Chrome's language

Switch DevTools to Portuguese

ElementsConsoleSourcesNetwork

topFilterDefault levelsNo issues

(index)	nome	ip	cpu	online
0	'srv-787-1'	'10.0.0.1'	35	true
1	'srv-787-2'	'10.0.0.2'	50	true
2	'srv-787-3'	'10.0.0.3'	60	true
3	'srv-787-4'	'10.0.0.4'	70	true
4	'srv-787-5'	'10.0.0.5'	75	true
5	'srv-787-6'	'10.0.0.6'	90	true
6	'srv-787-7'	'10.0.0.7'	0	false

Array(7)

srv-787-1script.js:12

35script.js:13

okscript.js:31

alertascript.js:32

criticascript.js:33

invalidoscript.js:34

29°C
Ensolarado

09:47
30/09/2026

Print 1 de 1 | image.png | anexado em 30/09/2026 09:47

Etapa 5: Trocando classes pelo JavaScript

Conceito: classList e eventos | Concluída em 30/09/2026 10:06

Fazer a interface reagir aos botões, mudando classes CSS em tempo de execução.

Conferência

- [x] Atualizar leituras sorteia novos valores e as cores dos cards mudam
- [x] O modo manutenção liga e desliga com o mesmo botão
- [x] O texto do botão muda conforme o estado

Pergunta

Explique a diferença entre `classList.add`, `remove`, `toggle` e `contains`, indicando onde cada um aparece (ou poderia aparecer) no seu código.

O 'classList' manipula as classes CSS de um elemento HTML. O 'add' adiciona uma classe (usei no código com 'card.classList.add(classificação)' para dar cores aos cards). O 'remove' retira uma classe específica (poderia ser usado também para limpar as classes antigas de um card antes de o atualizar). O 'toggle' funciona como um interruptor, adicionando a classe se esta não existir e a removendo se estiver presente (usei em `document.body.classList.toggle('manutencao')`). Por último, o 'contains' verifica se o elemento possui a classe, devolvendo um valor booleano (usei no 'if' para saber qual texto colocar no botão de manutenção).

Cole os dois addEventListener completos

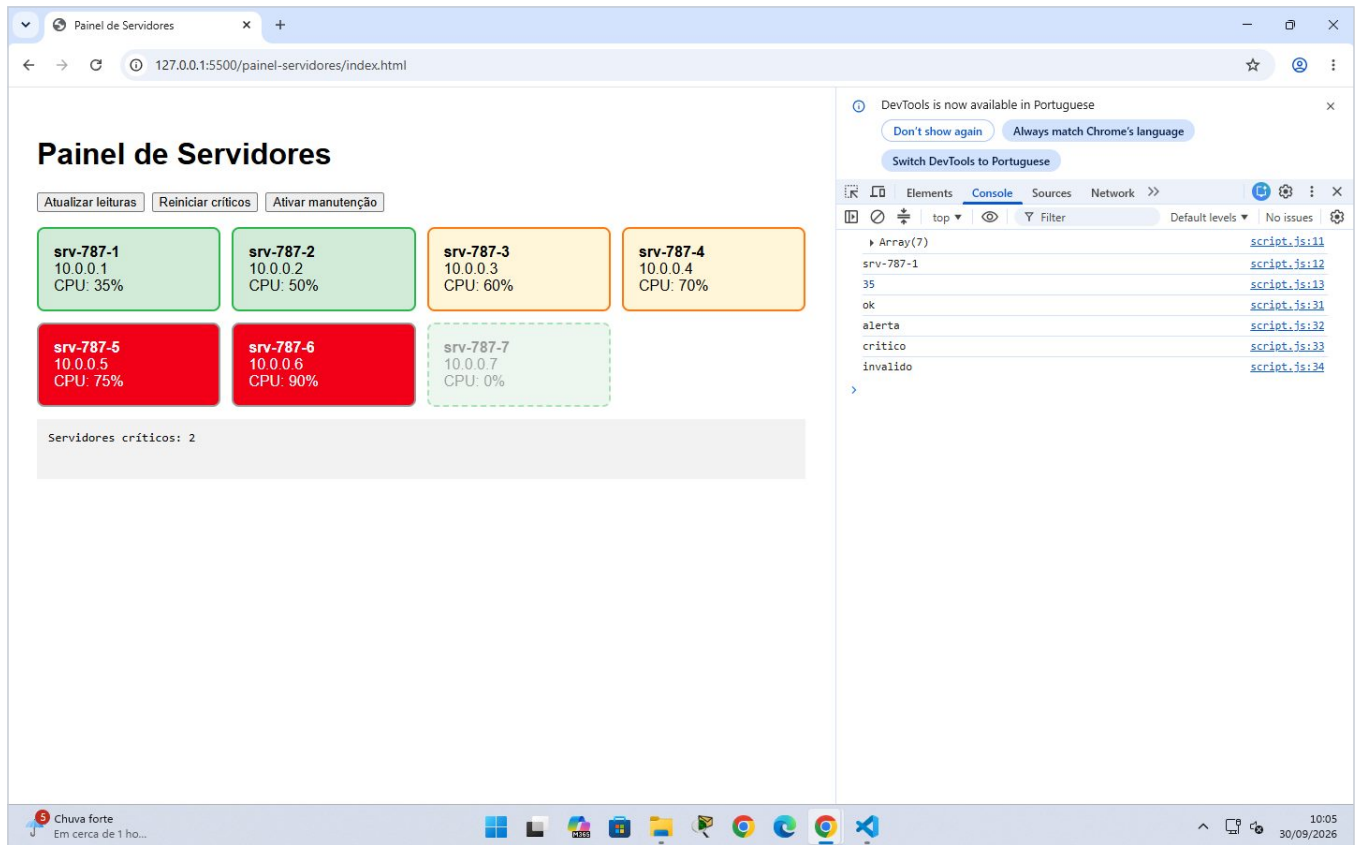
```
const btnAtualizar = document.getElementById("btn-atualizar");
const btnManutencao = document.getElementById("btn-manutencao");

btnAtualizar.addEventListener("click", function () {
  for (let i = 0; i < servidores.length; i++) {
    servidores[i].cpu = Math.floor(Math.random() * 101);
  }
  renderizar();
});

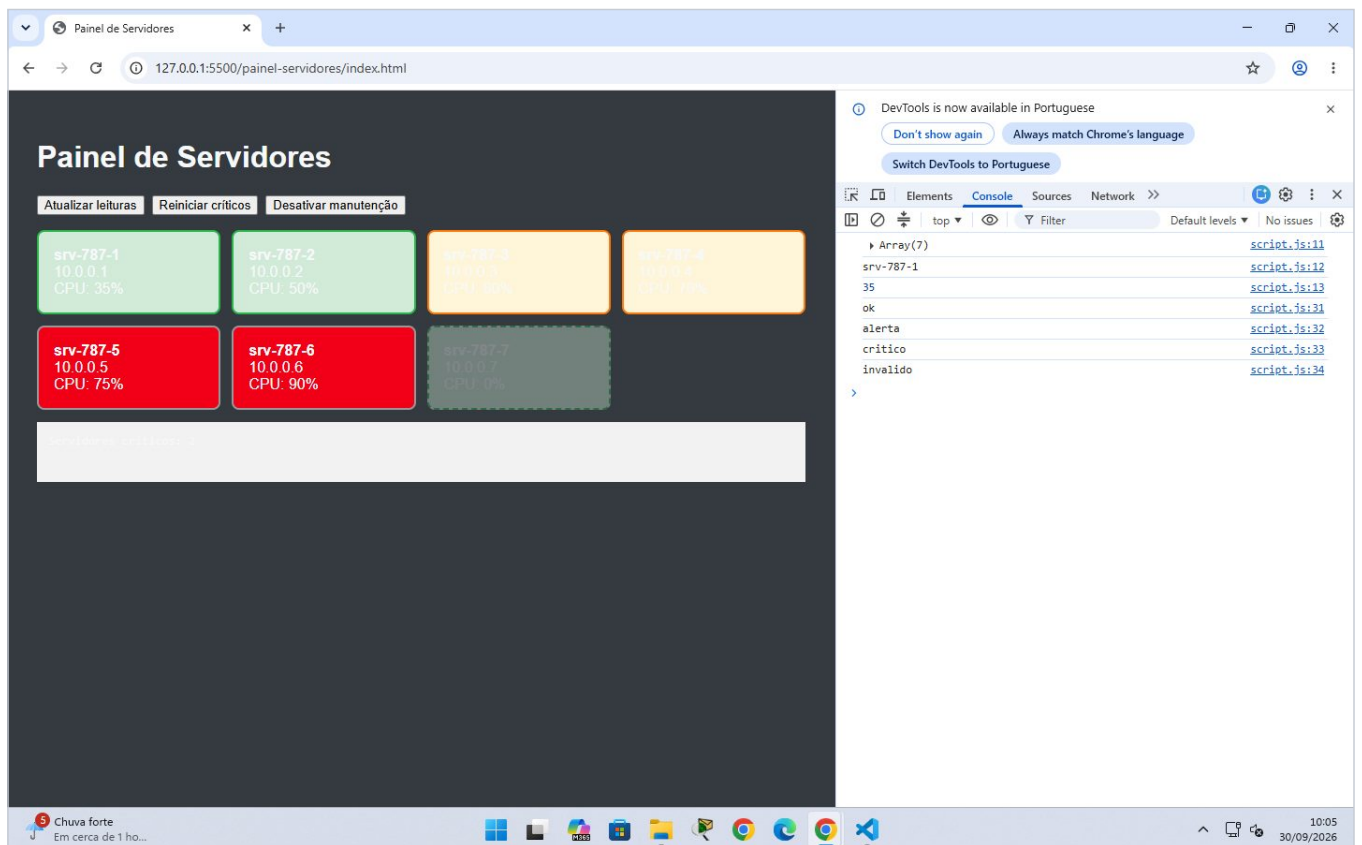
btnManutencao.addEventListener("click", function () {
  document.body.classList.toggle("manutencao");

  if (document.body.classList.contains("manutencao")) {
    btnManutencao.textContent = "Desativar manutenção";
  } else {
    btnManutencao.textContent = "Ativar manutenção";
  }
});
```

Prints



Print 1 de 2 | image.png | anexado em 30/09/2026 10:05



Print 2 de 2 | image.png | anexado em 30/09/2026 10:05

Etapa 6: Estabilizando servidores com while

Conceito: Laço while | Concluída em 30/09/2026 10:18

Reduzir a carga dos servidores críticos em 20 pontos por tentativa, repetindo até sair da faixa crítica ou atingir 5 tentativas.

Conferência

- [x] O while para quando a cpu sai da faixa crítica ou quando chega a 5 tentativas
- [x] Servidores offline são ignorados
- [x] O log mostra quantas tentativas cada servidor precisou

Pergunta

Por que o while é mais adequado que o for neste problema? O que aconteceria se a condição tentativas < MAX_TENTATIVAS fosse removida e REDUCAO valesse 0?

O while é mais adequado pra esse caso porque não sabemos previamente quantas reduções serão necessárias para fazer o servidor sair do estado crítico, mas sabemos que a condição de paragem é a CPU chegar abaixo de 75.

O for é melhor para contar um número exato de vezes. Se a condição 'tentativas < MAX_TENTATIVAS' fosse removida e a constante REDUCAO valesse 0, cairíamos num loop infinito, já que a CPU nunca ficaria abaixo de 75, a condição do while seria sempre verdadeira, e o programa bloquearia o navegador.

Cole a função reiniciarCriticos completa

```
const REDUCAO = 20;
const MAX_TENTATIVAS = 5;
const btnReiniciar = document.getElementById("btn-reiniciar");

function reiniciarCriticos() {
  const mensagens = [];

  for (let i = 0; i < servidores.length; i++) {
    const s = servidores[i];
    if (!s.online) continue;

    let tentativas = 0;
    while (s.cpu >= LIMITE_CRITICO && tentativas < MAX_TENTATIVAS) {
      s.cpu = s.cpu - REDUCAO;
      tentativas++;
    }

    if (tentativas > 0) {
      mensagens.push(s.nome + ": estabilizado após " + tentativas + " tentativa(s), cpu " + s.cpu + "%");
    }
  }

  renderizar();
  log.textContent = mensagens.length > 0 ? mensagens.join("\n") : "Nenhum servidor crítico.";
}

btnReiniciar.addEventListener("click", reiniciarCriticos);
```

Prints

Painel de Servidores

Atualizar leiturasReiniciar críticosAtivar manutenção

srv-787-110.0.0.1CPU: 78%

srv-787-210.0.0.2CPU: 83%

srv-787-310.0.0.3CPU: 6%

srv-787-410.0.0.4CPU: 66%

srv-787-510.0.0.5CPU: 23%

srv-787-610.0.0.6CPU: 95%

srv-787-710.0.0.7CPU: 31%

Servidores críticos: 3

DevTools is now available in Portuguese

Don't show againAlways match Chrome's language

Switch DevTools to Portuguese

ElementsConsoleSourcesNetwork

topFilter

Default levelsNo issues

(index)	nome	ip	cpu	online
0	'srv-787-1'	'10.0.0.1'	35	true
1	'srv-787-2'	'10.0.0.2'	50	true
2	'srv-787-3'	'10.0.0.3'	60	true
3	'srv-787-4'	'10.0.0.4'	70	true
4	'srv-787-5'	'10.0.0.5'	75	true
5	'srv-787-6'	'10.0.0.6'	90	true
6	'srv-787-7'	'10.0.0.7'	0	false

Array(7)

srv-787-1script.js:12

35script.js:13

okscript.js:31

alertascript.js:32

criticoscript.js:33

invalidoscript.js:34

30°CEnsolarado

10:1730/09/2026

Print 1 de 2 | image.png | anexado em 30/09/2026 10:17

Painel de Servidores

Atualizar leiturasReiniciar críticosAtivar manutenção

srv-787-110.0.0.1CPU: 58%

srv-787-210.0.0.2CPU: 63%

srv-787-310.0.0.3CPU: 6%

srv-787-410.0.0.4CPU: 66%

srv-787-510.0.0.5CPU: 23%

srv-787-610.0.0.6CPU: 55%

srv-787-710.0.0.7CPU: 31%

srv-787-1: estabilizado após 1 tentativa(s), cpu 58%
srv-787-2: estabilizado após 1 tentativa(s), cpu 63%
srv-787-6: estabilizado após 2 tentativa(s), cpu 55%

DevTools is now available in Portuguese

Don't show againAlways match Chrome's language

Switch DevTools to Portuguese

ElementsConsoleSourcesNetwork

topFilter

Default levelsNo issues

(index)	nome	ip	cpu	online
0	'srv-787-1'	'10.0.0.1'	35	true
1	'srv-787-2'	'10.0.0.2'	50	true
2	'srv-787-3'	'10.0.0.3'	60	true
3	'srv-787-4'	'10.0.0.4'	70	true
4	'srv-787-5'	'10.0.0.5'	75	true
5	'srv-787-6'	'10.0.0.6'	90	true
6	'srv-787-7'	'10.0.0.7'	0	false

Array(7)

srv-787-1script.js:12

35script.js:13

okscript.js:31

alertascript.js:32

criticoscript.js:33

invalidoscript.js:34

30°CEnsolarado

10:1730/09/2026

Print 2 de 2 | image.png | anexado em 30/09/2026 10:17

Etapa 7: Integração e entrega final

Conceito: Revisão geral | Concluída em 30/09/2026 10:34

Juntar tudo em um único script.js funcional e registrar o painel completo em funcionamento.

Conferência

- [x] O script.js final reúne todas as etapas e funciona sem erros
- [x] Testei os três botões em sequência
- [x] Revisei as evidências das etapas anteriores

Pergunta

Descreva um erro real que você encontrou durante a atividade (mensagem do console ou comportamento inesperado) e como você o resolveu.

O 'erro' que encontrei na verdade foi um problema com cores. Ao testar o botão 'Ativar manutenção', o body recebia o fundo escuro e o texto claro, mas os cartões '.ok' e '.alerta' e a área de #log herdavam a cor de texto clara sobre o fundo também claro, ficando impossível ler. Decidi editar o css forçando a propriedade color diretamente nas três classes pra evitar a herança de cor do body, garantindo a leitura correta.

Link informado

<https://github.com/Sonnyeros/apfront>

Cole o script.js completo

```
body { font-family: sans-serif; padding: 24px; }
.acoes { display: flex; gap: 8px; margin-bottom: 16px; }
#painel {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(170px, 1fr));
  gap: 12px;
}
.card { padding: 16px; border-radius: 8px; border: 2px solid #999; }

.ok {
  background-color: #d2ebd8;
  border-color: #29b64a;
  color: #212529
}
.alerta {
  background-color: #fff6d9;
  border-color: #ff790c;
  color: #212529
}
.critico {
  background-color: #f10018;
  color: white;
}
.offline { opacity: 0.4; border-style: dashed; }

body.manutencao {
  background-color: #343a40;
  color: #f8f9fa;
}
#log {
  background: #f2f2f2; padding: 12px; min-height: 40px;
  color: #212529
}
```

Prints

Painel de Servidores

127.0.0.1:5500/painel-servidores/index.html

Painel de Servidores

Atualizar leituras

Reiniciar críticos

Desativar manutenção

srv-787-1

10.0.0.1

CPU: 32%

srv-787-2

10.0.0.2

CPU: 32%

srv-787-3

10.0.0.3

CPU: 12%

srv-787-4

10.0.0.4

CPU: 31%

srv-787-5

10.0.0.5

CPU: 62%

srv-787-6

10.0.0.6

CPU: 56%

srv-787-7

10.0.0.7

CPU: 20%

srv-787-6: estabilizado após 2 tentativa(s), cpu 56%

DevTools is now available in Portuguese

Don't show again

Always match Chrome's language

Switch DevTools to Portuguese

Elements

Console

Sources

Network

>>

top

Filter

Default levels

No issues

(index)	nome	ip	cpu	online
0	'srv-787-1'	'10.0.0.1'	35	true
1	'srv-787-2'	'10.0.0.2'	50	true
2	'srv-787-3'	'10.0.0.3'	60	true
3	'srv-787-4'	'10.0.0.4'	70	true
4	'srv-787-5'	'10.0.0.5'	75	true
5	'srv-787-6'	'10.0.0.6'	90	true
6	'srv-787-7'	'10.0.0.7'	0	false

Array(7)

srv-787-1

35

ok

alerta

critico

invalido

30°C

Ensolarado

Windows

Taskbar icons

System tray

10:34

30/09/2026

Print 1 de 1 | image.png | anexado em 30/09/2026 10:34

Sonny Heros Campos Cardoso | Matrícula 202510787 | Verificação AE4F8F8C5BA8B9AD

15 / 15